

setTimeout(0ms) vs fetch().then() — Which Runs First?

Short Answer

The fetch's **.then() callback runs first**, even though setTimeout has a delay of 0 milliseconds. This is because of how the JavaScript event loop prioritizes different kinds of asynchronous callbacks: Promise callbacks (microtasks) always run before setTimeout callbacks (macrotasks/timer tasks), regardless of the timer delay value.

Why: Microtasks vs Macrotasks

JavaScript's event loop maintains two separate queues for callbacks that are not part of the main synchronous code:

- **Microtask queue** — holds Promise callbacks (.then, .catch, .finally) and async/await continuations.
- **Macrotask queue** (a.k.a. task queue) — holds setTimeout, setInterval, I/O callbacks, and UI events.

After every single macrotask finishes (including the initial run of the main script), the event loop always fully drains the entire microtask queue before picking up the next macrotask. This rule is fixed and does not depend on timer values — even a setTimeout with a delay of 0ms still has to wait its turn in the macrotask queue, and microtasks are always emptied first.

Step-by-Step Walkthrough

Consider this code:

```
setTimeout(() => {
  console.log('setTimeout callback');
}, 0);

fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => {
    console.log('fetch callback');
  });

console.log('main script end');
```

1. The synchronous code runs first.

setTimeout registers its callback in the macrotask queue (with a minimum 0ms delay, but it still has to wait). fetch() starts the network request and immediately returns a pending Promise — its .then() callback is registered to run once that Promise resolves. Then console.log('main script end') runs immediately, since it is plain synchronous code.

Output so far: **'main script end'** is printed first.

2. Once the call stack is empty, the event loop checks the microtask queue first.

If the fetch has already resolved (or resolves very quickly) by this point, its `.then()` callback is a microtask, so it gets executed before the event loop ever looks at the macrotask queue (where `setTimeout`'s callback is waiting).

Output: **'fetch callback'** is printed second.

3. Only after the microtask queue is completely empty does the event loop move to the next macrotask.

Now `setTimeout`'s callback finally gets its turn.

Output: **'setTimeout callback'** is printed last.

Final Order

1. main script end
2. fetch callback
3. setTimeout callback

Important Caveat

In real-world conditions, `fetch()` almost never resolves instantly — it depends on actual network latency, which is typically several milliseconds at minimum, so in practice the `setTimeout(0)` callback frequently fires before a real fetch resolves. The question's premise ('fetch resolves immediately') is a simplified/theoretical scenario used to test understanding of the event loop. Even under that idealized assumption, the rule still holds: as soon as both callbacks are ready to run, the microtask (fetch's `.then`) is always drained before the macrotask (`setTimeout`), because of how the event loop is specified to operate — not because of timing or speed.